PROJECT BASED EXPERIMENTAL LEARNING

COURSE- DATABASE MANAGEMENT SYSTEM (NoSQL / MongoDB)

PROJECT REPORT

TOPIC- MONGODB STUDENT DATABASE OPERATIONS



SUBMITTED TO- MR. Harendra Singh Rajpoot

Submitted by- Ashmit Pradhan

BCADS27

Roll no. - 1250258145

3rd semester

Babu Banarasi Das UniversityLucknow, U.P

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor Mr. Harendra Singh Rajpoot, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to IBM, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

Abstract

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A "students" collection is used to store student records containing roll number, name, age, department and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as $gt, $lt, $eq, $gte, $ne, $and, $or and $not. Every operation is documented with the exact command and a screenshot of the output produced in the MongoDB shell (mongosh). The implementation demonstrates how MongoDB can be used to efficiently manage, filter and retrieve structured student data.

Introduction

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 50 students across the AIML, Robotics, FinTech and Biotech departments (15 AIML, 12 Robotics, 11 FinTech and 12 Biotech at the time of insertion), and demonstrates the standard database operations - insertion, updation, deletion and querying - that form the backbone of any real-world database-driven application.

Methodology

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a database created for this purpose. The complete methodology, along with the exact code executed and a screenshot of the output produced by MongoDB at each step, is given below.

1. Insert One

Inserts a single student document (roll number 401) into the students collection.

Code:

```
db.students.insertOne({
  rollNo: 401,
  name: "Advait",
  age: 20,
  department: "AIML",
  marks: 74
})
```

Output:

```
> db.students.insertOne({ rollNo: 401, name: "Advait", age: 20, department: "AIML", marks: 74 })
{
  acknowledged: true,
  insertedId: ObjectId('6ad1be03dd73cf256d8f4d3e')
}
```

Figure 1: Output of insertOne()

2. Insert Many

Inserts multiple student documents (roll numbers 402 to 450) into the collection at the same time.

Code:

```
db.students.insertMany([
  { rollNo: 402, name: "Samaira", age: 21, department: "Robotics", marks: 76 },
  { rollNo: 403, name: "Rehaan", age: 20, department: "FinTech", marks: 69 },
  { rollNo: 404, name: "Kritika", age: 19, department: "Biotech", marks: 84 },
  { rollNo: 405, name: "Vivaan", age: 21, department: "AIML", marks: 53 },
  { rollNo: 406, name: "Anika", age: 21, department: "Robotics", marks: 62 },
  { rollNo: 407, name: "Zayan", age: 19, department: "FinTech", marks: 88 },
  { rollNo: 408, name: "Ritvi", age: 18, department: "Biotech", marks: 53 },
  { rollNo: 409, name: "Shlok", age: 19, department: "AIML", marks: 84 },
  { rollNo: 410, name: "Tanishka", age: 19, department: "Robotics", marks: 85 },
  { rollNo: 411, name: "Aarush", age: 20, department: "FinTech", marks: 59 },
  { rollNo: 412, name: "Nandini", age: 19, department: "Biotech", marks: 97 },
  { rollNo: 413, name: "Ishan", age: 19, department: "AIML", marks: 96 },
  { rollNo: 414, name: "Diksha", age: 19, department: "Robotics", marks: 80 },
  { rollNo: 415, name: "Ranveer", age: 20, department: "AIML", marks: 73 },
  { rollNo: 416, name: "Jhanvi", age: 18, department: "Biotech", marks: 72 },
  { rollNo: 417, name: "Tejas", age: 19, department: "AIML", marks: 35 },
  { rollNo: 418, name: "Anvika", age: 20, department: "Robotics", marks: 95 },
  { rollNo: 419, name: "Kunj", age: 20, department: "FinTech", marks: 90 },
  { rollNo: 420, name: "Mehak", age: 21, department: "Biotech", marks: 77 },
  { rollNo: 421, name: "Pranav", age: 20, department: "AIML", marks: 99 },
  { rollNo: 422, name: "Sanya", age: 19, department: "FinTech", marks: 74 },
  { rollNo: 423, name: "Vansh", age: 21, department: "FinTech", marks: 31 },
  { rollNo: 424, name: "Eshaan", age: 21, department: "Biotech", marks: 57 },
  { rollNo: 425, name: "Rhea", age: 21, department: "AIML", marks: 68 },
  { rollNo: 426, name: "Lavanya", age: 19, department: "Robotics", marks: 91 },
  { rollNo: 427, name: "Ojas", age: 19, department: "FinTech", marks: 70 },
  { rollNo: 428, name: "Charvi", age: 21, department: "Biotech", marks: 69 },
  { rollNo: 429, name: "Hardik", age: 18, department: "AIML", marks: 85 },
  { rollNo: 430, name: "Naisha", age: 21, department: "Robotics", marks: 79 },
  { rollNo: 431, name: "Manan", age: 21, department: "FinTech", marks: 82 },
  { rollNo: 432, name: "Pratiksha", age: 19, department: "Biotech", marks: 83 },
  { rollNo: 433, name: "Jivika", age: 19, department: "AIML", marks: 58 },
  { rollNo: 434, name: "Rishabh", age: 21, department: "Robotics", marks: 78 },
  { rollNo: 435, name: "Alvira", age: 21, department: "FinTech", marks: 66 },
  { rollNo: 436, name: "Tarun", age: 20, department: "Robotics", marks: 97 },
  { rollNo: 437, name: "Bhumi", age: 19, department: "AIML", marks: 84 },
  { rollNo: 438, name: "Chirag", age: 21, department: "Robotics", marks: 38 },
  { rollNo: 439, name: "Sejal", age: 20, department: "FinTech", marks: 55 },
  { rollNo: 440, name: "Harshit", age: 19, department: "Biotech", marks: 58 },
  { rollNo: 441, name: "Roshni", age: 21, department: "AIML", marks: 85 },
  { rollNo: 442, name: "Yug", age: 18, department: "Robotics", marks: 99 },
  { rollNo: 443, name: "Kashvi", age: 19, department: "Biotech", marks: 68 },
  { rollNo: 444, name: "Mayank", age: 21, department: "Biotech", marks: 83 },
  { rollNo: 445, name: "Disha", age: 19, department: "AIML", marks: 79 },
  { rollNo: 446, name: "Nirvaan", age: 20, department: "Robotics", marks: 67 },
  { rollNo: 447, name: "Shruti", age: 19, department: "FinTech", marks: 79 },
  { rollNo: 448, name: "Aniket", age: 18, department: "Biotech", marks: 84 },
  { rollNo: 449, name: "Gauri", age: 21, department: "AIML", marks: 92 },
  { rollNo: 450, name: "Madhav", age: 21, department: "AIML", marks: 79 }
])
```

Output:

```
> db.students.insertMany([ { rollNo: 402, ... }, ... { rollNo: 450, ... } ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ad1be03dd73cf256d8f4d3f'),
    '1': ObjectId('6ad1be03dd73cf256d8f4d40'),
    '2': ObjectId('6ad1be03dd73cf256d8f4d41'),
    '3': ObjectId('6ad1be03dd73cf256d8f4d42'),
    '4': ObjectId('6ad1be03dd73cf256d8f4d43'),
    // ... 44 more entries ('5' to '48') not shown
  }
}
```

Figure 2: Output of insertMany() - the shell returns one ObjectId per inserted document (list shortened here)

3. Update One

Updates the first document that matches the given condition - here, the marks of roll number 401 are updated from 74 to 81.

Code:

```
db.students.updateOne(
  { rollNo: 401 },
  { $set: { marks: 81 } }
)
```

Output:

```
> db.students.updateOne({ rollNo: 401 }, { $set: { marks: 81 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

Figure 3: Output of updateOne()

4. Update Many

Updates all documents that match the given condition - here, every student in the Robotics department is given a new status field set to "Active".

Code:

```
db.students.updateMany(
  { department: "Robotics" },
  { $set: { status: "Active" } }
)
```

Output:

```
> db.students.updateMany({ department: "Robotics" }, { $set: { status: "Active" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 12,
  modifiedCount: 12,
  upsertedCount: 0
}
```

Figure 4: Output of updateMany() - all 12 Robotics documents modified

Verification of the updates:

```
db.students.findOne({ rollNo: 401 })
```

Output:

```
> db.students.findOne({ rollNo: 401 })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3e'),
  rollNo: 401,
  name: 'Advait',
  age: 20,
  department: 'AIML',
  marks: 81
}
```

Figure 5: Roll no. 401 now shows marks: 81

5. Delete One

Deletes the first document that matches the given condition - here, the record of roll number 450 is removed.

Code:

```
db.students.deleteOne({ rollNo: 450 })
```

Output:

```
> db.students.deleteOne({ rollNo: 450 })
{
  acknowledged: true,
  deletedCount: 1
}
```

Figure 6: Output of deleteOne()

6. Delete Many

Deletes all documents that match the given condition - here, every student with marks less than 40 is removed. Three students matched: roll 417 (35 marks), roll 423 (31 marks) and roll 438 (38 marks).

Code:

```
db.students.deleteMany({ marks: { $lt: 40 } })
```

Output:

```
> db.students.deleteMany({ marks: { $lt: 40 } })
{
  acknowledged: true,
  deletedCount: 3
}
```

Figure 7: Output of deleteMany() - deletedCount is 3

Checking that the deleted records are gone:

```
db.students.find({ rollNo: { $in: [417, 423, 438, 450] } })
```

Output:

```
> db.students.find({ rollNo: { $in: [417, 423, 438, 450] } })
// no documents - all three were deleted
```

Figure 8: No document is returned for the three deleted roll numbers

Total documents after all update and delete operations:

```
db.students.countDocuments()
```

Output:

```
> db.students.countDocuments()
46
```

Figure 9: 46 documents remain (50 inserted - 1 - 3 deleted)

7. Query Operations

A series of find() queries were run using MongoDB's comparison and logical operators to filter and retrieve student records. For each query, the screenshot shows the shell output (the first three matching documents are captured; the shell prints the rest in the same format) and the table lists every matching record.

Marks greater than 80:

Code:

```
db.students.find({ marks: { $gt: 80 } })
```

Output:

```
> db.students.find({ marks: { $gt: 80 } })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3e'),
  rollNo: 401,
  name: 'Advait',
  age: 20,
  department: 'AIML',
  marks: 81
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d41'),
  rollNo: 404,
  name: 'Kritika',
  age: 19,
  department: 'Biotech',
  marks: 84
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d44'),
  rollNo: 407,
  name: 'Zayan',
  age: 19,
  department: 'FinTech',
  marks: 88
}
```

Figure 10: Marks greater than 80 - 21 documents matched (first 3 shown)

All matching records (21):

Roll No	Name	Age	Dept	Marks
401	Advait	20	AIML	81
404	Kritika	19	Biotech	84
407	Zayan	19	FinTech	88
409	Shlok	19	AIML	84
410	Tanishka	19	Robotics	85
412	Nandini	19	Biotech	97
413	Ishan	19	AIML	96
418	Anvika	20	Robotics	95
419	Kunj	20	FinTech	90
421	Pranav	20	AIML	99
426	Lavanya	19	Robotics	91

Roll No	Name	Age	Dept	Marks
429	Hardik	18	AIML	85
431	Manan	21	FinTech	82
432	Pratiksha	19	Biotech	83
436	Tarun	20	Robotics	97
437	Bhumi	19	AIML	84
441	Roshni	21	AIML	85
442	Yug	18	Robotics	99
444	Mayank	21	Biotech	83
448	Aniket	18	Biotech	84
449	Gauri	21	AIML	92

Marks less than 80:

Code:

```
db.students.find({ marks: { $lt: 80 } })
```

Output:

```
> db.students.find({ marks: { $lt: 80 } })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3f'),
  rollNo: 402,
  name: 'Samaira',
  age: 21,
  department: 'Robotics',
  marks: 76,
  status: 'Active'
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d40'),
  rollNo: 403,
  name: 'Rehaan',
  age: 20,
  department: 'FinTech',
  marks: 69
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d42'),
  rollNo: 405,
  name: 'Vivaan',
  age: 21,
  department: 'AIML',
  marks: 53
}
```

Figure 11: Marks less than 80 - 24 documents matched (first 3 shown)

All matching records (24):

Roll No	Name	Age	Dept	Marks
402	Samaira	21	Robotics	76
403	Rehaan	20	FinTech	69
405	Vivaan	21	AIML	53
406	Anika	21	Robotics	62
408	Ritvi	18	Biotech	53
411	Aarush	20	FinTech	59
415	Ranveer	20	AIML	73
416	Jhanvi	18	Biotech	72
420	Mehak	21	Biotech	77
422	Sanya	19	FinTech	74
424	Eshaan	21	Biotech	57
425	Rhea	21	AIML	68

Roll No	Name	Age	Dept	Marks
427	Ojas	19	FinTech	70
428	Charvi	21	Biotech	69
430	Naisha	21	Robotics	79
433	Jivika	19	AIML	58
434	Rishabh	21	Robotics	78
435	Alvira	21	FinTech	66
439	Sejal	20	FinTech	55
440	Harshit	19	Biotech	58
443	Kashvi	19	Biotech	68
445	Disha	19	AIML	79
446	Nirvaan	20	Robotics	67
447	Shruti	19	FinTech	79

Marks equal to 85:

Code:

```
db.students.find({ marks: { $eq: 85 } })
```

Output:

```
> db.students.find({ marks: { $eq: 85 } })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d47'),
  rollNo: 410,
  name: 'Tanishka',
  age: 19,
  department: 'Robotics',
  marks: 85,
  status: 'Active'
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d5a'),
  rollNo: 429,
  name: 'Hardik',
  age: 18,
  department: 'AIML',
  marks: 85
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d66'),
  rollNo: 441,
  name: 'Roshni',
  age: 21,
  department: 'AIML',
  marks: 85
}
```

Figure 12: Marks equal to 85 - 3 documents matched (first 3 shown)

All matching records (3):

Roll No	Name	Age	Dept	Marks
410	Tanishka	19	Robotics	85
429	Hardik	18	AIML	85
441	Roshni	21	AIML	85

Marks greater than or equal to 80:

Code:

```
db.students.find({ marks: { $gte: 80 } })
```

Output:

```
> db.students.find({ marks: { $gte: 80 } })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3e'),
  rollNo: 401,
  name: 'Advait',
  age: 20,
  department: 'AIML',
  marks: 81
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d41'),
  rollNo: 404,
  name: 'Kritika',
  age: 19,
  department: 'Biotech',
  marks: 84
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d44'),
  rollNo: 407,
  name: 'Zayan',
  age: 19,
  department: 'FinTech',
  marks: 88
}
```

Figure 13: Marks greater than or equal to 80 - 22 documents matched (first 3 shown)

All matching records (22):

Roll No	Name	Age	Dept	Marks
401	Advait	20	AIML	81
404	Kritika	19	Biotech	84
407	Zayan	19	FinTech	88
409	Shlok	19	AIML	84
410	Tanishka	19	Robotics	85
412	Nandini	19	Biotech	97
413	Ishan	19	AIML	96
414	Diksha	19	Robotics	80
418	Anvika	20	Robotics	95
419	Kunj	20	FinTech	90
421	Pranav	20	AIML	99

Roll No	Name	Age	Dept	Marks
426	Lavanya	19	Robotics	91
429	Hardik	18	AIML	85
431	Manan	21	FinTech	82
432	Pratiksha	19	Biotech	83
436	Tarun	20	Robotics	97
437	Bhumi	19	AIML	84
441	Roshni	21	AIML	85
442	Yug	18	Robotics	99
444	Mayank	21	Biotech	83
448	Aniket	18	Biotech	84
449	Gauri	21	AIML	92

Marks not equal to 85:

Code:

```
db.students.find({ marks: { $ne: 85 } })
```

Output:

```
> db.students.find({ marks: { $ne: 85 } })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3e'),
  rollNo: 401,
  name: 'Advait',
  age: 20,
  department: 'AIML',
  marks: 81
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3f'),
  rollNo: 402,
  name: 'Samaira',
  age: 21,
  department: 'Robotics',
  marks: 76,
  status: 'Active'
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d40'),
  rollNo: 403,
  name: 'Rehaan',
  age: 20,
  department: 'FinTech',
  marks: 69
}
```

Figure 14: Marks not equal to 85 - 43 documents matched (first 3 shown)

All matching records (43):

Roll No	Name	Age	Dept	Marks
401	Advait	20	AIML	81
402	Samaira	21	Robotics	76
403	Rehaan	20	FinTech	69
404	Kritika	19	Biotech	84
405	Vivaan	21	AIML	53
406	Anika	21	Robotics	62
407	Zayan	19	FinTech	88
408	Ritvi	18	Biotech	53
409	Shlok	19	AIML	84
411	Aarush	20	FinTech	59
412	Nandini	19	Biotech	97
413	Ishan	19	AIML	96
414	Diksha	19	Robotics	80
415	Ranveer	20	AIML	73
416	Jhanvi	18	Biotech	72
418	Anvika	20	Robotics	95
419	Kunj	20	FinTech	90
420	Mehak	21	Biotech	77
421	Pranav	20	AIML	99
422	Sanya	19	FinTech	74
424	Eshaan	21	Biotech	57
425	Rhea	21	AIML	68

Roll No	Name	Age	Dept	Marks
426	Lavanya	19	Robotics	91
427	Ojas	19	FinTech	70
428	Charvi	21	Biotech	69
430	Naisha	21	Robotics	79
431	Manan	21	FinTech	82
432	Pratiksha	19	Biotech	83
433	Jivika	19	AIML	58
434	Rishabh	21	Robotics	78
435	Alvira	21	FinTech	66
436	Tarun	20	Robotics	97
437	Bhumi	19	AIML	84
439	Sejal	20	FinTech	55
440	Harshit	19	Biotech	58
442	Yug	18	Robotics	99
443	Kashvi	19	Biotech	68
444	Mayank	21	Biotech	83
445	Disha	19	AIML	79
446	Nirvaan	20	Robotics	67
447	Shruti	19	FinTech	79
448	Aniket	18	Biotech	84
449	Gauri	21	AIML	92

Marks greater than 70 AND age less than 21:

Code:

```
db.students.find({
  $and: [
    { marks: { $gt: 70 } },
    { age: { $lt: 21 } }
  ]
})
```

Output:

```
> db.students.find({
    $and: [
      { marks: { $gt: 70 } },
      { age: { $lt: 21 } }
    ]
  })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3e'),
  rollNo: 401,
  name: 'Advait',
  age: 20,
  department: 'AIML',
  marks: 81
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d41'),
  rollNo: 404,
  name: 'Kritika',
  age: 19,
  department: 'Biotech',
  marks: 84
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d44'),
  rollNo: 407,
  name: 'Zayan',
  age: 19,
  department: 'FinTech',
  marks: 88
}
```

Figure 15: Marks greater than 70 AND age less than 21 - 23 documents matched (first 3 shown)

All matching records (23):

Roll No	Name	Age	Dept	Marks
401	Advait	20	AIML	81
404	Kritika	19	Biotech	84
407	Zayan	19	FinTech	88
409	Shlok	19	AIML	84
410	Tanishka	19	Robotics	85
412	Nandini	19	Biotech	97
413	Ishan	19	AIML	96
414	Diksha	19	Robotics	80
415	Ranveer	20	AIML	73
416	Jhanvi	18	Biotech	72
418	Anvika	20	Robotics	95
419	Kunj	20	FinTech	90
421	Pranav	20	AIML	99
422	Sanya	19	FinTech	74
426	Lavanya	19	Robotics	91
429	Hardik	18	AIML	85
432	Pratiksha	19	Biotech	83
436	Tarun	20	Robotics	97
437	Bhumi	19	AIML	84
442	Yug	18	Robotics	99
445	Disha	19	AIML	79
447	Shruti	19	FinTech	79
448	Aniket	18	Biotech	84

Department is AIML OR FinTech:

Code:

```
db.students.find({
  $or: [
    { department: "AIML" },
    { department: "FinTech" }
  ]
})
```

Output:

```
> db.students.find({
    $or: [
      { department: "AIML" },
      { department: "FinTech" }
    ]
  })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3e'),
  rollNo: 401,
  name: 'Advait',
  age: 20,
  department: 'AIML',
  marks: 81
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d40'),
  rollNo: 403,
  name: 'Rehaan',
  age: 20,
  department: 'FinTech',
  marks: 69
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d42'),
  rollNo: 405,
  name: 'Vivaan',
  age: 21,
  department: 'AIML',
  marks: 53
}
```

Figure 16: Department is AIML OR FinTech - 23 documents matched (first 3 shown)

All matching records (23):

Roll No	Name	Age	Dept	Marks
401	Advait	20	AIML	81
403	Rehaan	20	FinTech	69
405	Vivaan	21	AIML	53
407	Zayan	19	FinTech	88
409	Shlok	19	AIML	84
411	Aarush	20	FinTech	59
413	Ishan	19	AIML	96
415	Ranveer	20	AIML	73
419	Kunj	20	FinTech	90
421	Pranav	20	AIML	99
422	Sanya	19	FinTech	74
425	Rhea	21	AIML	68

Roll No	Name	Age	Dept	Marks
427	Ojas	19	FinTech	70
429	Hardik	18	AIML	85
431	Manan	21	FinTech	82
433	Jivika	19	AIML	58
435	Alvira	21	FinTech	66
437	Bhumi	19	AIML	84
439	Sejal	20	FinTech	55
441	Roshni	21	AIML	85
445	Disha	19	AIML	79
447	Shruti	19	FinTech	79
449	Gauri	21	AIML	92

Marks NOT greater than 80:

Code:

```
db.students.find({ marks: { $not: { $gt: 80 } } })
```

Output:

```
> db.students.find({ marks: { $not: { $gt: 80 } } })
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d3f'),
  rollNo: 402,
  name: 'Samaira',
  age: 21,
  department: 'Robotics',
  marks: 76,
  status: 'Active'
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d40'),
  rollNo: 403,
  name: 'Rehaan',
  age: 20,
  department: 'FinTech',
  marks: 69
}
{
  _id: ObjectId('6ad1be03dd73cf256d8f4d42'),
  rollNo: 405,
  name: 'Vivaan',
  age: 21,
  department: 'AIML',
  marks: 53
}
```

Figure 17: Marks NOT greater than 80 - 25 documents matched (first 3 shown)

All matching records (25):

Roll No	Name	Age	Dept	Marks
402	Samaira	21	Robotics	76
403	Rehaan	20	FinTech	69
405	Vivaan	21	AIML	53
406	Anika	21	Robotics	62
408	Ritvi	18	Biotech	53
411	Aarush	20	FinTech	59
414	Diksha	19	Robotics	80
415	Ranveer	20	AIML	73
416	Jhanvi	18	Biotech	72
420	Mehak	21	Biotech	77
422	Sanya	19	FinTech	74
424	Eshaan	21	Biotech	57
425	Rhea	21	AIML	68

Roll No	Name	Age	Dept	Marks
427	Ojas	19	FinTech	70
428	Charvi	21	Biotech	69
430	Naisha	21	Robotics	79
433	Jivika	19	AIML	58
434	Rishabh	21	Robotics	78
435	Alvira	21	FinTech	66
439	Sejal	20	FinTech	55
440	Harshit	19	Biotech	58
443	Kashvi	19	Biotech	68
445	Disha	19	AIML	79
446	Nirvaan	20	Robotics	67
447	Shruti	19	FinTech	79

8. MongoDB Operator Summary

Operator	Meaning	Example
$gt	Greater than	marks: { $gt: 80 }
$lt	Less than	marks: { $lt: 80 }
$eq	Equal to	marks: { $eq: 85 }
$gte	Greater than or equal to	marks: { $gte: 80 }
$ne	Not equal to	marks: { $ne: 85 }
$and	All conditions must be true	marks > 70 AND age < 21
$or	At least one condition must be true	AIML OR FinTech
$not	Negates a condition	NOT (marks > 80)

Total dataset: 50 students inserted (Roll No. 401-450); 46 remain after deleting Roll No. 450 and the three students with marks below 40.

Implementation Details

- The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh).
- A single collection named "students" is used, with fields: rollNo, name, age, department and marks.
- insertOne() and insertMany() are used to populate the collection with 50 student records.
- updateOne() and updateMany() with the $set operator are used to modify existing fields and add new ones (e.g. status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.
- find() is combined with comparison operators ($gt, $lt, $eq, $gte, $ne) and logical operators ($and, $or, $not) to retrieve specific subsets of student data.

- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

Results

All CRUD operations executed successfully on the students collection. The insertOne and insertMany operations populated the collection with 50 student records. The updateOne operation successfully changed one student's marks (74 to 81), while updateMany correctly modified all 12 Robotics-department documents to add an "Active" status. The deleteOne operation removed roll number 450, and deleteMany removed the 3 records with marks below 40 (roll 417 (35 marks), roll 423 (31 marks) and roll 438 (38 marks)), leaving 46 documents in the collection. The query operations correctly filtered students by marks, age and department using comparison and logical operators: 21 students scored above 80 while 25 did not, 24 students scored below 80 while 22 scored 80 or more, 3 students scored exactly 85, and 23 students belong to the AIML or FinTech departments. These counts are consistent with each other (21 + 25 = 46 and 24 + 22 = 46), demonstrating that MongoDB's query engine returns accurate, condition-based results.

Conclusion

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

Future Scope

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.
- Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

References

- MongoDB Official Documentation: https://www.mongodb.com/docs/
- MongoDB CRUD Operations: https://www.mongodb.com/docs/manual/crud/

- MongoDB Query Operators: https://www.mongodb.com/docs/manual/reference/operator/query/
- mongosh (MongoDB Shell) Documentation: https://www.mongodb.com/docs/mongodb-shell/

THANK YOU!